Same Bsts

AlgoExpert · Hard · Binary Search Trees

The problem

- Would inserting the two arrays (left to right) produce identical BSTs, without building them?
- Recurse on first-smaller / first-greater-or-equal descendants within value bounds, using indices only.

Intuition

- Two arrays give the same BST if they share the root and their smaller and larger elements build the same subtrees.

Brute force vs optimal

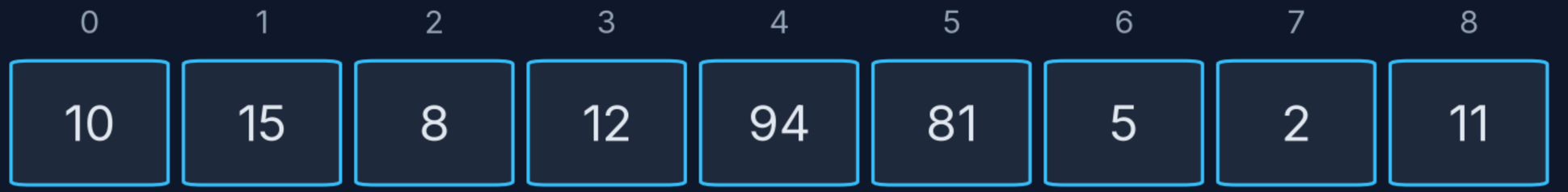
- Build both BSTs and compare → $O(n^2)$ worst
- Split into smaller/larger lists recursively → $O(n^2)$ time and space
- Index-based recursion with bounds → $O(n^2)$ time, $O(d)$ space

Approach

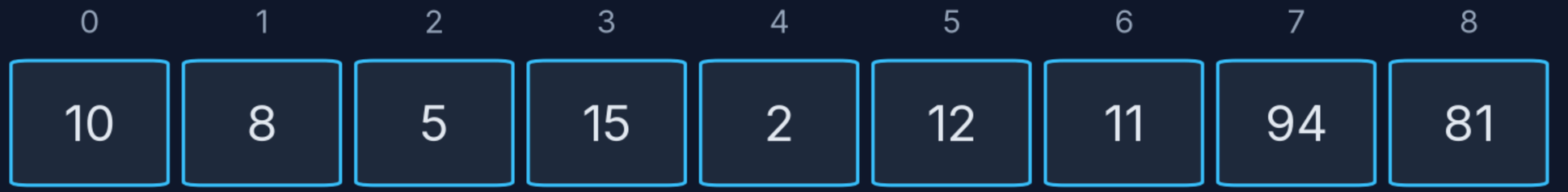
- 1. Compare first elements and lengths
- 2. Split each into smaller and bigger-or-equal lists keeping order
- 3. Recurse on both halves

Worked example

array A



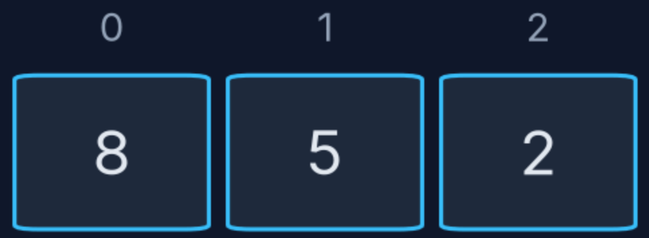
array B



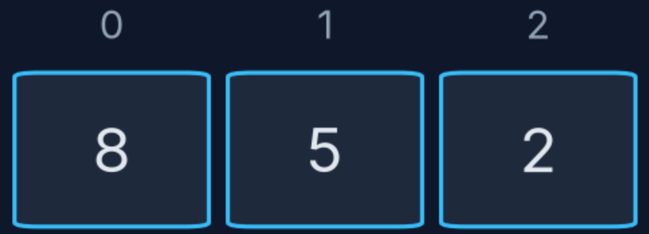
Same root? 10 = 10 ✓

Worked example

A: smaller than 10



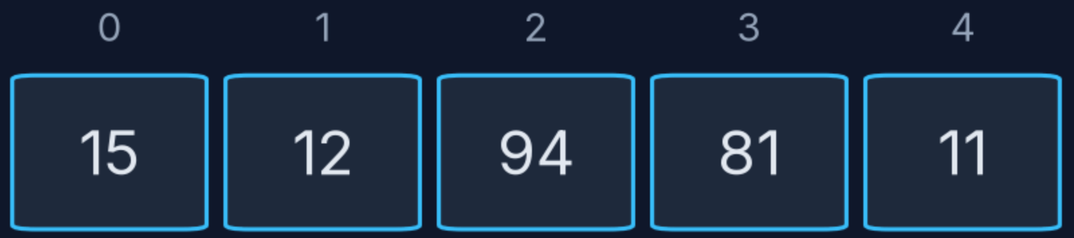
B: smaller than 10



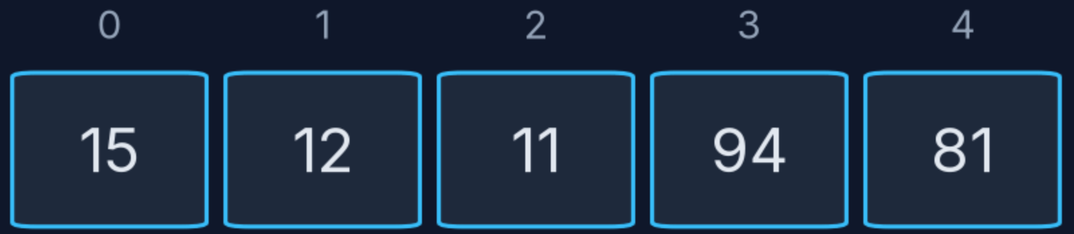
Left subtrees: [8, 5, 2] vs [8, 5, 2]

Worked example

A: ≥ 10

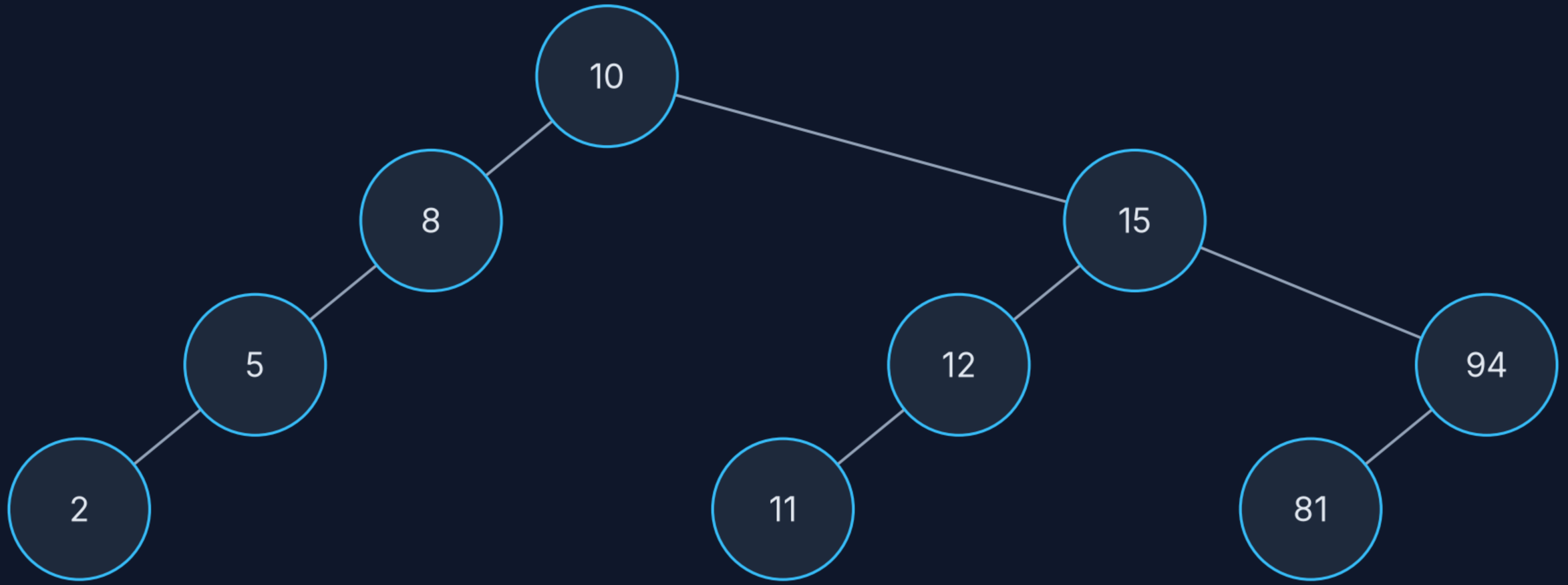


B: ≥ 10



Right subtrees: first is 15 in both

Worked example



Same BST

Java solution (1/2)

```java
public class SameBsts {
    public static boolean sameBsts(List<Integer> a, List<Integer> b) {
        if (a.size() != b.size()) return false;
        if (a.isEmpty()) return true;
        return same(a, b, 0, 0, Integer.MIN_VALUE, Integer.MAX_VALUE);
    }

    private static boolean same(List<Integer> a, List<Integer> b, int ia, int ib, int min, int max) {
        if (ia == -1 || ib == -1) return ia == ib;
        if (!a.get(ia).equals(b.get(ib))) return false;
        int v = a.get(ia);
        return same(a, b, smaller(a, ia, min), smaller(b, ib, min), min, v)
                && same(a, b, bigger(a, ia, max), bigger(b, ib, max), v, max);
    }

    private static int smaller(List<Integer> x, int start, int min) {
        for (int i = start + 1; i < x.size(); i++) if (x.get(i) < x.get(start) && x.get(i) >= min) return i;
        return -1;
    }
```

Java solution (2/2)

```
    private static int bigger(List<Integer> x, int start, int max) {
        for (int i = start + 1; i < x.size(); i++) if (x.get(i) >= x.get(start) && x.get(i) < max) return i;
        return -1;
    }
}
```

Complexity

- Time: O(n^2)
- Space: O(d)

Edge cases & pitfalls

- Duplicates go right (≥)
- Different lengths → false
- Empty arrays → true

Interview tips & follow-ups

- Explain why relative order within subtrees matters

Recap

- Two arrays give the same BST if they share the root and their smaller and larger elements build the same subtrees.
- Complexity: O(n^2)